

WEEK-6

Task-1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

CODE:

```
root@DESKTOP-QU0HVJI: ~/! X + v
GNU nano 7.2 history.c *
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 100

int main()
{
    char history[MAX][SIZE];
    int count = 0;
    char command[SIZE];

    printf("Command History\n");
    printf("Type 'history' to display commands\n");
    printf("Type 'exit' to stop\n");

    while (1)
    {
        printf("Enter command: ");

        fgets(command, SIZE, stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0)
        {
            printf("\n--- History ---\n");

            for (int i = 0; i < count; i++)
                printf("%d: %s\n", i + 1, history[i]);

            printf("-----\n");
            continue;
        }

        if (count < MAX)
        {
            strcpy(history[count], command);
            count++;
        }
        else
        {
            // Remove oldest command
            for (int i = 0; i < MAX - 1; i++)
                strcpy(history[i], history[i + 1]);

            strcpy(history[MAX - 1], command);
        }
    }

    printf("History program stopped.\n");

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL# cd SKILL 0
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# nano history.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# gcc history.c -o history
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./hisstory
-bash: ./hisstory: No such file or directory
root@DESKTOP-QU0HVJI:~/SKILL/skill-6#
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./history
Command History
Type 'history' to display commands
Type 'exit' to stop
Enter command: ls
Enter command: pwd
Enter command: date
Enter command: exit
History program stopped.
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# |
```

Report: The command history buffer was successfully implemented in Ubuntu.

The program stores commands in a fixed-size buffer and displays them when requested.

When the buffer becomes full, the oldest command is removed to store the new command.

The history was tested with multiple commands and the capacity limit was verified.

Task-2 To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

Code:

```
root@DESKTOP-QU0HVJI: ~/! × + v
GNU nano 7.2 pipeline.c *
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 10
#define SIZE 100

int main()
{
    char input[SIZE];
    char commands[MAX_COMMANDS][SIZE];
    int count = 0;

    printf("Enter pipeline: ");
    fgets(input, SIZE, stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS)
    {
        while (*token == ' ')
            token++;

        strcpy(commands[count], token);
        count++;

        token = strtok(NULL, "|");
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < count; i++)
        printf("Process %d: %s\n", i + 1, commands[i]);

    printf("\nTotal processes: %d\n", count);

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# nano pipeline.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# gcc pipeline.c -o pipeline
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./pipeline
Enter pipeline: ls | grep txt | sort

Pipeline Structure:
Process 1: ls
Process 2: grep txt
Process 3: sort

Total processes: 3
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# |
```

Report: The pipeline structure program was successfully implemented in Ubuntu.

It accepts multiple commands separated by the pipe (|) symbol.

The commands are stored and displayed in their execution order.

The pipeline layout was tested using multiple connected commands.

TASK-3: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

Code:

```
root@DESKTOP-QU0HVJI: ~/! × + v
GNU nano 7.2 pipe.c *
#include <sys/wait.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == 0)
    {
        // First process: ls
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == 0)
    {
        // Second process: wc -l
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    return 0;
}
```

OUTPUT:

```
pipe.c:42:9: note: include <stdlib.h> or provide a declaration of 'exit'
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# nano pipe.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# gcc pipe.c -o pipe
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./ pipe
-bash: ./: Is a directory
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./pipe
6
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# |
```

Report: The pipe communication program was successfully implemented in Ubuntu.

A pipe was created to transfer the output of one process to another process.

fork() was used to create child processes and dup2() was used for stream redirection.

The processes were synchronized using waitpid() and the unused descriptors were closed.

TASK-4: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

Code:

```
root@DESKTOP-QU0HVJI: ~/? × + ▾
GNU nano 7.2 multi_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    // Process 1: ls
    if (fork() == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);
        exit(1);
    }

    // Process 2: grep .c
    if (fork() == 0)
    {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);
        exit(1);
    }

    // Process 3: wc -l
    if (fork() == 0)
    {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# nano multi_pipe.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# gcc multi_pipe.c -o multi_pipe
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# ./multi_pipe
4
root@DESKTOP-QU0HVJI:~/SKILL/skill-6# |
```

Report: The multiple-pipe program was successfully implemented in Ubuntu. Multiple processes were created using `fork()` and connected using pipes. `dup2()` was used to redirect the input and output streams between processes. All unused file descriptors were closed and the parent process waited for the child processes to complete.